

Cash Flow Platform - Product Architecture & Implementation Plan

Objective

Transform the existing cash flow prediction model into an enterprise financial management platform that provides continuous, derived financial insights rather than basic input-to-output inference.

Current System

1. A hybrid Long Short-Term Memory (LSTM) and dense neural network trained on the synthetic SME Kaggle dataset.
2. A lightweight FastAPI application serving the model for inference.
3. A basic React form application where users manually input financial data to receive predictions.

Functional Requirements

1. Automated Data Ingestion

To monitor cash flow continuously and predict future outcomes, the platform must eliminate manual data entry.

1. Build an automated ingestion layer that integrates directly with a single target accounting software (e.g., QuickBooks Online or Xero) via OAuth2.
2. Support standardized Excel and CSV spreadsheet uploads for legacy financial records.
3. Store all ingested ledger records in a relational database to serve as the single source of truth for all derived feature calculations.

2. Covenant Health Dashboard

Upgrade the manual form into an executive dashboard for managing overall organizational health.

1. Center the dashboard on a Health Delta (ΔH), which quantifies the enterprise's financial well-being and resilience.
2. Pass actual ingested data through the model to establish a baseline cash flow prediction. Next, vary static balance sheet features specifically `credit_score`,

`current_assets` , and `capex` across a defined range of values and re-run inference.
Formula,

$$\Delta H = f(X_{seq}, X'_{static}) - f(X_{seq}, X_{static})$$

3. Receivables Optimization Engine

Quantify the exact volume of capital trapped by delayed client payments.

1. Generate a normal net cash flow prediction across a 6-month rolling window using standard temporal transaction data (X_{seq}).
2. Clone the sequence tensor and set `payment_delay` to `0.0` while holding all other features constant, then calculate the simulated cash flow.
3. Compute the mathematical difference between the baseline and simulated outputs for every client in the ledger. Rank the debtor list to flag the most notorious late-paying clients based on their downward pressure on organizational liquidity..

4. Liquidity Crisis Router

Autonomously prescribe mitigation strategies whenever the model forecasts an impending cash deficit.

1. Trigger an automated optimization loop whenever the baseline model's predicted net cash flow for the succeeding interval (net_flow_{t+1}) breaches a customized organizational solvency buffer.
2. Extract pre-computed 64-dimensional intermediate embeddings (`seq_feat` and `static_feat`) from the neural network to prevent system latency. Isolate and manipulate discretionary outflow variables, specifically targeting scheduled capital expenditures (`capex`) and flexible debt obligations like `monthly_repayment` schedules.
3. Feed synthetic permutations of these discretionary outflows directly into the model's isolated dense fusion block (`fast-fusion` endpoint). The loop tests specific operational trade-offs until it converges on a state where:

$$\hat{y}_{optimized} \geq Solvency\ Buffer$$

4. Present a prioritized, mathematical action plan (e.g., 1. Postpone Equipment Purchase by 18 days; 2. Restructure Short-Term Loan Repayment), providing executives with toggle switches to commit these adjustments directly into the active operational budget.

Phase-wise Implementation Plan

Phase 1 - Infrastructure Setup & Data Persistence

1. Deploy a PostgreSQL database using Docker.
2. Implement asynchronous database connection pooling and session management using SQLAlchemy 2.0 within FastAPI.
3. Construct normalized relational tables mapped directly to the neural network's input requirements:
 - `client_enterprises` : Stores core registration details (`company_reg_number`), turnover levels, and employee size tiers.
 - `static_health_metrics` : Houses the 10 non-sequential parameters required for the dense neural branch: `capex` , `cogs` , `current_assets` , `current_liabilities` , `fixed_assets` , `long_term_liabilities` , `credit_score` , `failure_score` , `debt_to_revenue_ratio` , and `missed_payments_number` .
 - `temporal_transactions` : Houses granular historical records aggregated into monthly time-series buckets for the 5 temporal features: `total_invoice_amount` , `payment_delay` , `monthly_repayment` , `total_inflows` , and `total_outflows` .
 - `simulation_checkpoints` : A dedicated state table to store ephemeral simulation runs without polluting live financial ledgers.

Phase 2 - Data Ingestion & Automation Layer

1. Build an ingestion module to authenticate with a primary accounting software platform and establish webhooks for automatic ledger updates.
2. Develop a Pandas-based parser to ingest and standardize CSV and Excel file uploads into the PostgreSQL schema.
3. Implement strict Pydantic v2 schemas at the API boundary to enforce type coercion, handle null imputation (e.g., defaulting missing credit card payments to zero), and structure raw data for tensor conversion.
4. Build an in-memory data transformation pipeline natively within FastAPI to automatically transform raw ingested ledgers into 6-month sliding window sequence tensors (X_{seq}) and rolling static balance sheet vectors (X_{static}) during the HTTP request lifecycle.

Phase 3 - Model Serving & Simulation Microservice

Add the following core service endpoints -

1. `POST /api/v1/predict/baseline` : Accepts client registration identifiers, applies standardization scaling, and executes a standard forward pass to return the baseline next-month net liquidity (net_flow_{t+1}).
2. `POST /api/v1/simulate/health-delta` : Freezes temporal sequences and executes parameter sweeps across static balance sheet variables (`credit_score` , `current_assets` , `capex`) to compute the Health Delta (ΔH).
3. `POST /api/v1/simulate/trapped-liquidity` : Clones client temporal tensors, applies a zero-mask to the `payment_delay` index, and outputs the cash recovery delta.
4. `POST /api/v1/simulate/fast-fusion` : Accepts pre-computed intermediate embeddings (`seq_feat` and `static_feat`), bypassing recurrent LSTM layers for sub-millisecond scenario testing.

Phase 4 - Business Logic & Orchestration

1. Construct a scheduled backend service that evaluates client metrics against credit covenants and calculates sensitivity curves using the Health Delta endpoint.
2. Build a synchronous batch algorithm that processes accounts receivable ledgers, scores debtors via the trapped liquidity endpoint, and outputs a ranked action queue of notorious late payers.
3. Implement a deterministic orchestration loop anchored to the PostgreSQL `simulation_checkpoints` table. Configure it to iterate through discretionary outflow reduction permutations within the application runtime until it identifies a valid operational pathway back to solvency.

Phase 5 - Frontend Platform & Integration

1. Upgrade the existing basic input form into an enterprise dashboard focused on real-time monitoring and interactive financial planning.
2. Integrate the dashboard with backend API.